

Importing libraries

```
In [ ]: import nbconvert
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np # Linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns
from keras.models import Sequential
from keras.layers import Dense, Dropout
import os
import warnings
from tensorflow import keras
from keras import layers
warnings.filterwarnings('ignore')
```

```
In [4]: !pip install -q -U keras-tuner
```

```
0.0/129.4 kB ? eta -:--:--
129.4/129.4 kB 4.4 MB/s eta 0:00:00
```

```
In [5]: from keras_tuner import RandomSearch
from keras_tuner import HyperParameters
```

Reading the data

```
In [6]: train_data = pd.read_csv('human_activity_train.csv')
test_data = pd.read_csv('human_activity_test.csv')

print(f'Shape of train data is: {train_data.shape}\nShape of test data is: {test_data.shape}')

Shape of train data is: (7352, 563)
Shape of test data is: (2947, 563)
```

Data analysis

```
In [7]: pd.set_option("display.max_columns", None)
train_data.head()
```

	tBodyAcc- mean()-X	tBodyAcc- mean()-Y	tBodyAcc- mean()-Z	tBodyAcc- std()-X	tBodyAcc- std()-Y	tBodyAcc- std()-Z	tBodyAcc- mad()-X	tBodyAcc- mad()-Y
0	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112	-0.983111
1	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807	-0.975300
2	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520	-0.967187
3	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099	-0.983403
4	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321	-0.979987

```
In [8]: train_data.columns
```

```
Out[8]: Index(['tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y', 'tBodyAcc-mean()-Z',
               'tBodyAcc-std()-X', 'tBodyAcc-std()-Y', 'tBodyAcc-std()-Z',
               'tBodyAcc-mad()-X', 'tBodyAcc-mad()-Y', 'tBodyAcc-mad()-Z',
               'tBodyAcc-max()-X',
               ...,
               'fBodyBodyGyroJerkMag-kurtosis()', 'angle(tBodyAccMean,gravity)',
               'angle(tBodyAccJerkMean,gravityMean)',
               'angle(tBodyGyroMean,gravityMean)',
               'angle(tBodyGyroJerkMean,gravityMean)', 'angle(X,gravityMean)',
               'angle(Y,gravityMean)', 'angle(Z,gravityMean)', 'subject', 'Activity'],
              dtype='object', length=563)
```

```
In [9]: train_data.describe()
```

Out[9]:

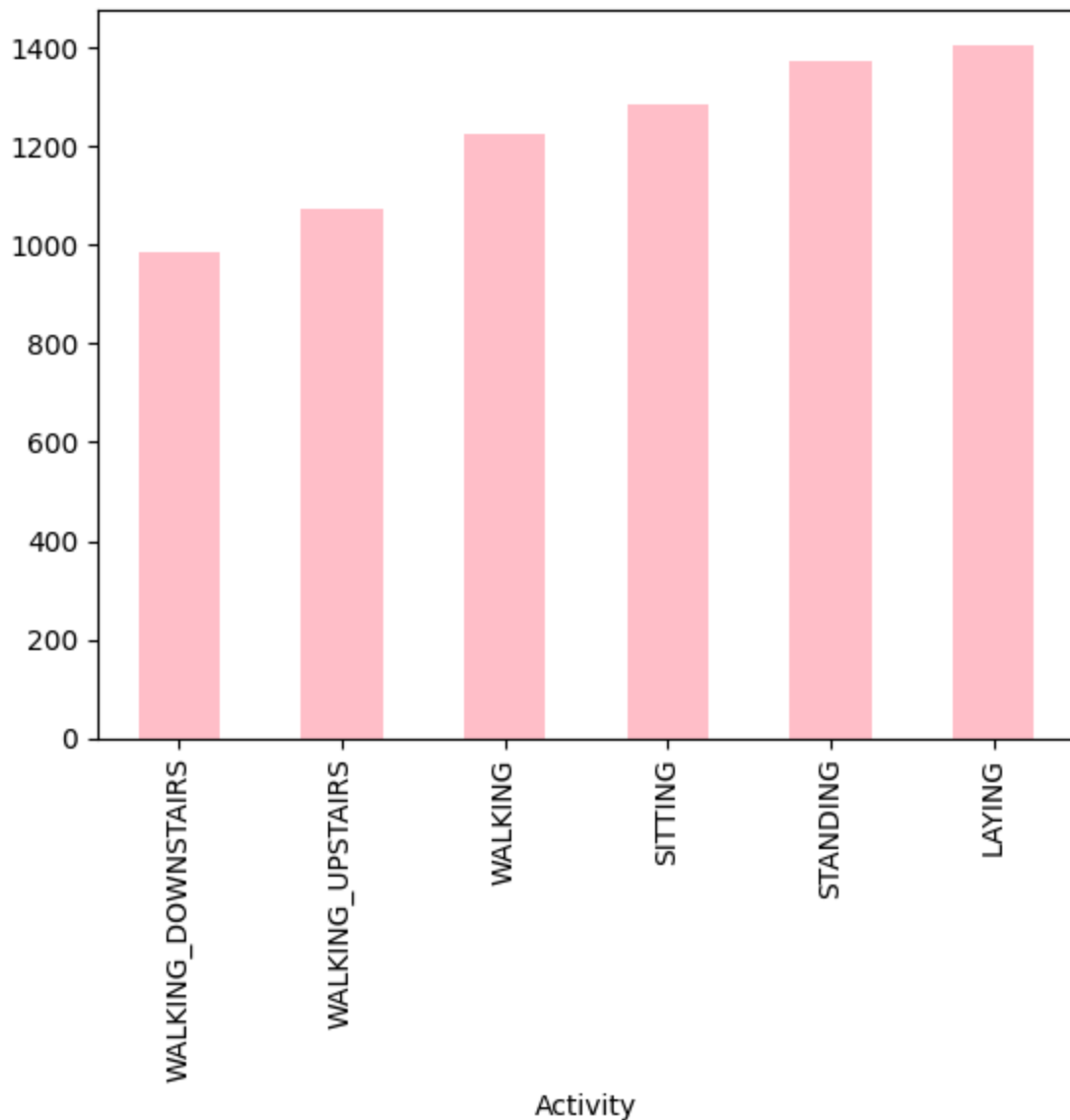
	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-count()
count	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352.000000	7352
mean	0.274488	-0.017695	-0.109141	-0.605438	-0.510938	-0.604754	-0
std	0.070261	0.040811	0.056635	0.448734	0.502645	0.418687	0
min	-1.000000	-1.000000	-1.000000	-1.000000	-0.999873	-1.000000	-1
25%	0.262975	-0.024863	-0.120993	-0.992754	-0.978129	-0.980233	-0
50%	0.277193	-0.017219	-0.108676	-0.946196	-0.851897	-0.859365	-0
75%	0.288461	-0.010783	-0.097794	-0.242813	-0.034231	-0.262415	-0
max	1.000000	1.000000	1.000000	1.000000	0.916238	1.000000	1

```
In [10]: train_data['Activity'].unique()
```

```
Out[10]: array(['STANDING', 'SITTING', 'LAYING', 'WALKING', 'WALKING_DOWNSTAIRS',
                'WALKING_UPSTAIRS'], dtype=object)
```

```
In [11]: train_data['Activity'].value_counts().sort_values().plot(kind = 'bar', color = 'pink')
```

```
Out[11]: <Axes: xlabel='Activity'>
```



Preparing train and test data

```
In [12]: x_train, y_train = train_data.iloc[:, :-2], train_data.iloc[:, -1:]  
x_test, y_test = test_data.iloc[:, :-2], test_data.iloc[:, -1:]  
x_train.shape, y_train.shape
```

```
Out[12]: ((7352, 561), (7352, 1))
```

```
In [13]: x_test, y_test = test_data.iloc[:, :-2], test_data.iloc[:, -1:]  
x_test.shape, y_test.shape
```

```
Out[13]: ((2947, 561), (2947, 1))
```

```
In [14]: le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.fit_transform(y_test)
x_test.shape, y_test.shape, x_train.shape, y_train.shape
```

```
Out[14]: ((2947, 561), (2947,), (7352, 561), (7352,))
```

```
In [15]: scaling_data = MinMaxScaler()
x_train = scaling_data.fit_transform(x_train)
x_test = scaling_data.transform(x_test)
```

Creating base model

```
In [16]: model = Sequential()
model.add(Dense(units=64,kernel_initializer='normal',activation='sigmoid',input_dim=561))
model.add(Dropout(0.2))
model.add(Dense(units=6,kernel_initializer='normal',activation='softmax'))
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])
history = model.fit(x_train, y_train, batch_size = 64, epochs= 10,validation_data = (x_test, y_test))
```

Epoch 1/10

115/115 ————— 4s 13ms/step - accuracy: 0.3912 - loss: 1.4992 - val_accuracy: 0.6990 - val_loss: 1.0351

Epoch 2/10

115/115 ————— 1s 4ms/step - accuracy: 0.7033 - loss: 0.9512 - val_accuracy: 0.8103 - val_loss: 0.7271

Epoch 3/10

115/115 ————— 0s 4ms/step - accuracy: 0.8087 - loss: 0.6655 - val_accuracy: 0.8636 - val_loss: 0.5386

Epoch 4/10

115/115 ————— 1s 4ms/step - accuracy: 0.8492 - loss: 0.4969 - val_accuracy: 0.8839 - val_loss: 0.4310

Epoch 5/10

115/115 ————— 1s 5ms/step - accuracy: 0.8868 - loss: 0.3872 - val_accuracy: 0.8812 - val_loss: 0.3621

Epoch 6/10

115/115 ————— 1s 4ms/step - accuracy: 0.9047 - loss: 0.3210 - val_accuracy: 0.9097 - val_loss: 0.2957

Epoch 7/10

115/115 ————— 1s 4ms/step - accuracy: 0.9098 - loss: 0.2805 - val_accuracy: 0.9111 - val_loss: 0.2724

Epoch 8/10

115/115 ————— 1s 7ms/step - accuracy: 0.9220 - loss: 0.2413 - val_accuracy: 0.9233 - val_loss: 0.2399

Epoch 9/10

115/115 ————— 1s 6ms/step - accuracy: 0.9192 - loss: 0.2258 - val_accuracy: 0.9270 - val_loss: 0.2248

Epoch 10/10

115/115 ————— 1s 8ms/step - accuracy: 0.9331 - loss: 0.2054 - val_accuracy: 0.9199 - val_loss: 0.2093

Hypertuning the model

```
In [17]: def build_model(hp):
model = keras.Sequential()
for i in range(hp.Int('num_layers', 2, 25)):
    model.add(layers.Dense(units = hp.Int('units' + str(i), min_value=32, max_v
                                kernel_initializer= hp.Choice('initializer', ['unifo
                                activation= hp.Choice('activation', ['relu', 'sigmoi
model.add(layers.Dense(6, kernel_initializer= hp.Choice('initializer', ['unifor
model.add(
    Dropout(0.2))
model.compile(
    optimizer = 'adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
return model

tuner = RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials= 5,
    executions_per_trial=3,
    directory='project', project_name = 'Human_activity_recognition')

tuner.search_space_summary()
```

```
Search space summary
Default search space size: 5
num_layers (Int)
{'default': None, 'conditions': [], 'min_value': 2, 'max_value': 25, 'step': 1, 'sam
pling': 'linear'}
units0 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step': 32,
'sampling': 'linear'}
initializer (Choice)
{'default': 'uniform', 'conditions': [], 'values': ['uniform', 'normal'], 'ordered':
False}
activation (Choice)
{'default': 'relu', 'conditions': [], 'values': ['relu', 'sigmoid', 'tanh'], 'ordere
d': False}
units1 (Int)
{'default': None, 'conditions': [], 'min_value': 32, 'max_value': 512, 'step': 32,
'sampling': 'linear'}
```

```
In [18]: tuner.search(x_train, y_train,
                    epochs= 10,
                    validation_data=(x_test, y_test))
```

```
Trial 5 Complete [00h 03m 22s]
val_accuracy: 0.18221920728683472
```

```
Best val_accuracy So Far: 0.9343965649604797
Total elapsed time: 00h 07m 52s
```

```
In [19]: tuner.results_summary()
```

Results summary
Results in project/Human_activity_recognition
Showing 10 best trials
Objective(name="val_accuracy", direction="max")

Trial 2 summary
Hyperparameters:
num_layers: 3
units0: 320
initializer: normal
activation: sigmoid
units1: 128
units2: 384
units3: 64
units4: 224
units5: 320
Score: 0.9343965649604797

Trial 1 summary
Hyperparameters:
num_layers: 3
units0: 96
initializer: normal
activation: relu
units1: 64
units2: 416
units3: 96
units4: 320
units5: 448
Score: 0.9282886385917664

Trial 3 summary
Hyperparameters:
num_layers: 6
units0: 32
initializer: normal
activation: relu
units1: 512
units2: 32
units3: 160
units4: 128
units5: 416
Score: 0.89582626024882

Trial 0 summary
Hyperparameters:
num_layers: 6
units0: 384
initializer: normal
activation: sigmoid
units1: 384
units2: 32
units3: 32
units4: 32
units5: 32
Score: 0.35052594542503357

Trial 4 summary
Hyperparameters:
num_layers: 23
units0: 480
initializer: uniform
activation: tanh
units1: 480
units2: 192
units3: 192
units4: 416
units5: 480
units6: 32
units7: 32
units8: 32
units9: 32
units10: 32
units11: 32
units12: 32
units13: 32
units14: 32
units15: 32
units16: 32
units17: 32
units18: 32
units19: 32
units20: 32
units21: 32
units22: 32
Score: 0.18221920728683472

```
In [20]: model=tuner.get_best_models(num_models=1)[0]  
         history = model.fit(x_train,y_train, epochs=51, validation_data=(x_test,y_test))
```

Epoch 1/51
230/230  3s 9ms/step - accuracy: 0.7685 - loss: 0.7071 - val_accuracy: 0.9342 - val_loss: 0.6614

Epoch 2/51
230/230  2s 8ms/step - accuracy: 0.7708 - loss: 0.7844 - val_accuracy: 0.9393 - val_loss: 0.5762

Epoch 3/51
230/230  2s 8ms/step - accuracy: 0.7872 - loss: 0.6599 - val_accuracy: 0.9186 - val_loss: 0.9575

Epoch 4/51
230/230  2s 11ms/step - accuracy: 0.7846 - loss: 0.6374 - val_accuracy: 0.9454 - val_loss: 0.4907

Epoch 5/51
230/230  2s 11ms/step - accuracy: 0.7798 - loss: 0.6538 - val_accuracy: 0.9260 - val_loss: 0.7419

Epoch 6/51
230/230  2s 8ms/step - accuracy: 0.7721 - loss: 0.7405 - val_accuracy: 0.9332 - val_loss: 0.6437

Epoch 7/51
230/230  2s 8ms/step - accuracy: 0.7728 - loss: 0.6254 - val_accuracy: 0.9403 - val_loss: 0.5878

Epoch 8/51
230/230  2s 8ms/step - accuracy: 0.7735 - loss: 0.6310 - val_accuracy: 0.9196 - val_loss: 0.9381

Epoch 9/51
230/230  2s 8ms/step - accuracy: 0.7713 - loss: 0.6852 - val_accuracy: 0.9040 - val_loss: 1.0788

Epoch 10/51
230/230  3s 10ms/step - accuracy: 0.7801 - loss: 0.6325 - val_accuracy: 0.9508 - val_loss: 0.5056

Epoch 11/51
230/230  3s 11ms/step - accuracy: 0.7788 - loss: 0.5427 - val_accuracy: 0.9413 - val_loss: 0.7237

Epoch 12/51
230/230  2s 8ms/step - accuracy: 0.7890 - loss: 0.5648 - val_accuracy: 0.9396 - val_loss: 0.7111

Epoch 13/51
230/230  3s 8ms/step - accuracy: 0.7935 - loss: 0.5763 - val_accuracy: 0.9386 - val_loss: 0.6368

Epoch 14/51
230/230  2s 8ms/step - accuracy: 0.7804 - loss: 0.5346 - val_accuracy: 0.9108 - val_loss: 1.0732

Epoch 15/51
230/230  2s 8ms/step - accuracy: 0.7853 - loss: 0.5690 - val_accuracy: 0.9118 - val_loss: 1.0969

Epoch 16/51
230/230  2s 9ms/step - accuracy: 0.7793 - loss: 0.6142 - val_accuracy: 0.9464 - val_loss: 0.7039

Epoch 17/51
230/230  3s 11ms/step - accuracy: 0.7731 - loss: 0.6565 - val_accuracy: 0.9528 - val_loss: 0.5270

Epoch 18/51
230/230  2s 8ms/step - accuracy: 0.7766 - loss: 0.6125 - val_accuracy: 0.9437 - val_loss: 0.6176

Epoch 19/51
230/230  3s 8ms/step - accuracy: 0.7838 - loss: 0.6019 - val_accuracy: 0.9437 - val_loss: 0.6176

uracy: 0.9508 - val_loss: 0.5182
Epoch 20/51
230/230  3s 8ms/step - accuracy: 0.7785 - loss: 0.5805 - val_acc
uracy: 0.9423 - val_loss: 0.6459
Epoch 21/51
230/230  2s 8ms/step - accuracy: 0.7851 - loss: 0.5863 - val_acc
uracy: 0.9430 - val_loss: 0.7090
Epoch 22/51
230/230  2s 11ms/step - accuracy: 0.7724 - loss: 0.6538 - val_acc
uracy: 0.9352 - val_loss: 0.8831
Epoch 23/51
230/230  2s 10ms/step - accuracy: 0.7829 - loss: 0.5734 - val_acc
uracy: 0.9539 - val_loss: 0.5425
Epoch 24/51
230/230  2s 8ms/step - accuracy: 0.7874 - loss: 0.5470 - val_acc
uracy: 0.9077 - val_loss: 1.6502
Epoch 25/51
230/230  2s 8ms/step - accuracy: 0.7791 - loss: 0.6770 - val_acc
uracy: 0.9481 - val_loss: 0.7060
Epoch 26/51
230/230  2s 8ms/step - accuracy: 0.7797 - loss: 0.5580 - val_acc
uracy: 0.9467 - val_loss: 0.7575
Epoch 27/51
230/230  2s 8ms/step - accuracy: 0.7905 - loss: 0.5647 - val_acc
uracy: 0.9291 - val_loss: 0.8711
Epoch 28/51
230/230  2s 10ms/step - accuracy: 0.8024 - loss: 0.5309 - val_acc
uracy: 0.9460 - val_loss: 0.5925
Epoch 29/51
230/230  3s 10ms/step - accuracy: 0.7930 - loss: 0.5025 - val_acc
uracy: 0.9277 - val_loss: 0.9160
Epoch 30/51
230/230  2s 8ms/step - accuracy: 0.7791 - loss: 0.5570 - val_acc
uracy: 0.9440 - val_loss: 0.7409
Epoch 31/51
230/230  2s 8ms/step - accuracy: 0.7736 - loss: 0.6156 - val_acc
uracy: 0.9423 - val_loss: 0.6788
Epoch 32/51
230/230  2s 8ms/step - accuracy: 0.8001 - loss: 0.4744 - val_acc
uracy: 0.9522 - val_loss: 0.5922
Epoch 33/51
230/230  2s 8ms/step - accuracy: 0.7809 - loss: 0.5174 - val_acc
uracy: 0.8843 - val_loss: 2.2815
Epoch 34/51
230/230  2s 8ms/step - accuracy: 0.7880 - loss: 0.5998 - val_acc
uracy: 0.9423 - val_loss: 0.7928
Epoch 35/51
230/230  3s 13ms/step - accuracy: 0.7812 - loss: 0.5389 - val_acc
uracy: 0.9281 - val_loss: 0.9756
Epoch 36/51
230/230  2s 8ms/step - accuracy: 0.7792 - loss: 0.5742 - val_acc
uracy: 0.9501 - val_loss: 0.6880
Epoch 37/51
230/230  2s 8ms/step - accuracy: 0.7846 - loss: 0.5918 - val_acc
uracy: 0.8829 - val_loss: 2.0174
Epoch 38/51

230/230 ————— 3s 8ms/step - accuracy: 0.8038 - loss: 0.5053 - val_accuracy: 0.9613 - val_loss: 0.5486
Epoch 39/51

230/230 ————— 2s 8ms/step - accuracy: 0.7954 - loss: 0.5126 - val_accuracy: 0.9264 - val_loss: 1.1140
Epoch 40/51

230/230 ————— 2s 9ms/step - accuracy: 0.7866 - loss: 0.5704 - val_accuracy: 0.9423 - val_loss: 0.8522
Epoch 41/51

230/230 ————— 3s 12ms/step - accuracy: 0.7951 - loss: 0.5134 - val_accuracy: 0.9315 - val_loss: 1.0271
Epoch 42/51

230/230 ————— 2s 8ms/step - accuracy: 0.7867 - loss: 0.5729 - val_accuracy: 0.9508 - val_loss: 0.7034
Epoch 43/51

230/230 ————— 2s 8ms/step - accuracy: 0.7880 - loss: 0.5731 - val_accuracy: 0.9294 - val_loss: 1.1507
Epoch 44/51

230/230 ————— 2s 8ms/step - accuracy: 0.7939 - loss: 0.5056 - val_accuracy: 0.9562 - val_loss: 0.6347
Epoch 45/51

230/230 ————— 2s 7ms/step - accuracy: 0.7795 - loss: 0.5669 - val_accuracy: 0.9501 - val_loss: 0.7515
Epoch 46/51

230/230 ————— 2s 8ms/step - accuracy: 0.7874 - loss: 0.5217 - val_accuracy: 0.9389 - val_loss: 0.8694
Epoch 47/51

230/230 ————— 2s 10ms/step - accuracy: 0.7939 - loss: 0.4925 - val_accuracy: 0.9175 - val_loss: 1.3486
Epoch 48/51

230/230 ————— 3s 14ms/step - accuracy: 0.8010 - loss: 0.5195 - val_accuracy: 0.9549 - val_loss: 0.6004
Epoch 49/51

230/230 ————— 2s 8ms/step - accuracy: 0.7897 - loss: 0.5222 - val_accuracy: 0.9467 - val_loss: 0.8146
Epoch 50/51

230/230 ————— 2s 10ms/step - accuracy: 0.7973 - loss: 0.5069 - val_accuracy: 0.9576 - val_loss: 0.5084
Epoch 51/51

230/230 ————— 2s 8ms/step - accuracy: 0.7857 - loss: 0.5206 - val_accuracy: 0.9274 - val_loss: 1.0953

```
In [21]: model.summary()
import tensorflow as tf

from tensorflow import keras

Callback = tf.keras.callbacks.EarlyStopping(monitor='accuracy', patience=3)
mo_fitt = model.fit(x_train,y_train, epochs=200, validation_data=(x_test,y_test), c
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 320)	179,840
dense_1 (Dense)	(None, 128)	41,088
dense_2 (Dense)	(None, 384)	49,536
dense_3 (Dense)	(None, 6)	2,310
dropout (Dropout)	(None, 6)	0

Total params: 818,324 (3.12 MB)

Trainable params: 272,774 (1.04 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 545,550 (2.08 MB)

Epoch 1/200

230/230 ————— 2s 8ms/step - accuracy: 0.7945 - loss: 0.5373 - val_accuracy: 0.9457 - val_loss: 0.7024

Epoch 2/200

230/230 ————— 3s 13ms/step - accuracy: 0.7858 - loss: 0.5044 - val_accuracy: 0.9511 - val_loss: 0.6589

Epoch 3/200

230/230 ————— 2s 8ms/step - accuracy: 0.8037 - loss: 0.4813 - val_accuracy: 0.9447 - val_loss: 0.7119

Epoch 4/200

230/230 ————— 2s 8ms/step - accuracy: 0.7938 - loss: 0.5536 - val_accuracy: 0.9328 - val_loss: 0.9740

Epoch 5/200

230/230 ————— 2s 8ms/step - accuracy: 0.7859 - loss: 0.5296 - val_accuracy: 0.9389 - val_loss: 0.7983

Epoch 6/200

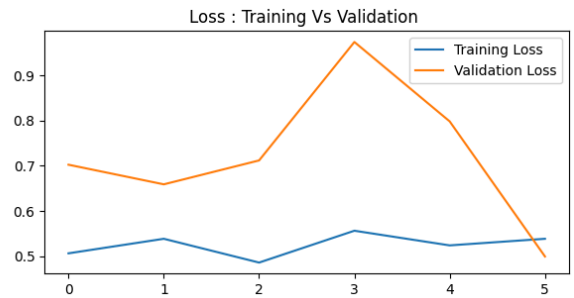
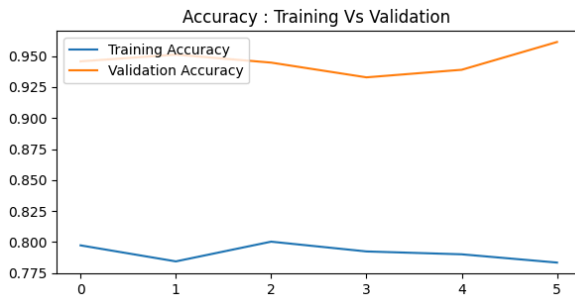
230/230 ————— 2s 8ms/step - accuracy: 0.7866 - loss: 0.5214 - val_accuracy: 0.9613 - val_loss: 0.4992

```
In [24]: accuracy = mo_fitt.history['accuracy']
loss = mo_fitt.history['loss']
validation_loss = mo_fitt.history['val_loss']
validation_accuracy = mo_fitt.history['val_accuracy']

plt.figure(figsize=(15, 7))
plt.subplot(2, 2, 1)
plt.plot(range(6), accuracy, label='Training Accuracy')
plt.plot(range(6), validation_accuracy, label='Validation Accuracy')
plt.legend(loc='upper left')
plt.title('Accuracy : Training Vs Validation ')

plt.subplot(2, 2, 2)
plt.plot(range(6), loss, label='Training Loss')
plt.plot(range(6), validation_loss, label='Validation Loss')
```

```
plt.title('Loss : Training Vs Validation ')\nplt.legend(loc='upper right')\nplt.show()
```



Documentation

Summary

In this project we are classifying human activity based on data captured by their smartphone.

For this project we used a different approach, instead of using the usual classification algorithms, we trained a neural network model to classify the data.

Implementation

First thing was to analyse the data. The dataset has measurements from the gyroscope and accelerometer in form of 3-axial linear acceleration and 3-axial angular velocity. It also has a 561-feature vector with time and frequency and the activity label.

The activity labels are the following: Standing, Sitting, Laying, Walking, Walking Downstairs, Walking Upstairs. We can also see the entries are decently balanced in terms of activity labels.

The next step is creating the base model. For this, a sequential model was used with a normal kernel initializer and with sigmoid and softmax activations. After training in 10 epochs it achieved 0.933 accuracy.

Then comes the hypertuning step. For this a random search is used, with different configurations for the kernel initializer and activation. This way we got a 0.934 accuracy for the best configuration: normal initializer with a sigmoid activation.

Results

As a results, after using the best configuration and retraining for 51 epochs, the accuracy went over 0.95.